Comprehensive Technical Project Documentation

Predictive Procurement Analytics System

Table of Contents

1. Introduction and Project Scope
2. High-Level System Architecture
3. Raw Data Dictionary (`new/Augmented/`)
4. ETL Pipeline Implementation (`etl_pipeline.py`)
5. Machine Learning Engine (`feature_engine.py`)
6. Decision Support Dashboard (`dashboard_app.py`)
7. Target Database Schema (`schema.sql`)
8. Deployment and Execution

---

1. Introduction and Project Scope

The **Predictive Procurement Analytics System** addresses a fundamental operational inefficiency in higher education: the over-ordering of physical textbooks and course materials. Historically, universities order textbooks assuming 100% of enrolled students will purchase them. In reality, students frequently opt-out of university-provided bundles due to high costs, utilizing secondary markets, renting, or pirating materials instead. This leads to massive sunk inventory costs.

This project implements a sophisticated data mining and machine learning pipeline to analyze historical transactions, student demographics, and product pricing. It predicts exactly which students are likely to opt-out, calculates the projected financial spend, and surfaces these insights to procurement executives via an interactive Streamlit dashboard.

---

2. High-Level System Architecture

The application is structured into a modular pipeline, separating data ingestion, predictive modeling, and user interface rendering:

1. **Storage / File System**: Raw historical datasets containing millions of student interactions split by university (FIU, EKU, Mercer, etc.) exist in the `new/Augmented/` directory.
2. **ETL (Extract, Transform, Load)**: The `etl_pipeline.py` script dynamically crawls the file system, ingests the data into Pandas dataframes, handles memory-safe sampling, and engineers dozens of derived semantic features.
3. **Machine Learning**: The `feature_engine.py` script consumes the ETL dataframe, isolates independent variables, trains a `RandomForestClassifier` in real-time, and scores every student row with a definitive `Predicted_Purchase_Prob` .
4. **Presentation**: The `dashboard_app.py` script serves a modern, dark-themed Streamlit web interface. It caches the heavy operations and uses Plotly Express to render interactive data visualizations.

---

3. Raw Data Dictionary

The data ingested from `new/Augmented/*_balanced_dataset.csv` contains granular transactional and demographic data.

Identity and Academic Context

- `sis_user_id` : Unique surrogate key representing a specific student.
- `section_id` : The exact course section the student is enrolled in (e.g., `BN-8304-1-126-A-23-421`).
- `term_code` : The semester identifier (e.g., `F` for Fall, `W` for Winter/Spring).
- `term_year` : The 2-digit academic year.
- `student_full_part_time_status` : `F` (Full-Time) or `P` (Part-Time).

Demographic and Financial Context

- `family_annual_income` : Numeric representation of household income, strongly correlating with price sensitivity.
- `has_loan` : Binary indicator (1/0) of whether the student relies on student loans.
- `has_scholarship` : Binary indicator (1/0) of whether the student possesses scholarship funding.
- `part_time_job` : Binary indicator outlining employment status.

Product (Textbook) Context

- `title` : Title of the textbook.
- `author` : Author of the textbook.
- `isbn` : Unique product identifier.
- `ebook_ind` : 1.0 if the product is an explicit digital access code or eBook.
- `is_rental` : Binary flag indicating if the specific transaction target is a rental item.
- `retail_new` : The brand-new purchase price of the textbook.
- `retail_used` : Secondary market textbook price.
- `retail_new_rent` : The price to rent a brand-new copy.

Derived Metrics (Pre-calculated in source DB)

- `price_affordability_score` : A proprietary metric representing the general financial strain of the purchase.
- `will_buy` : The **Target Variable**. 1 if the student actually purchased the bundle, 0 if they opted out.

---

4. ETL Pipeline Implementation (`etl_pipeline.py`)

The ETL module is solely responsible for turning the raw, noisy CSV files into a clean `mapped_df` that directly fuels the machine learning model and dashboard.

4.1 Memory-Safe Ingestion Strategy

The raw dataset is heavily fragmented and totals gigabytes of data. Using default `pd.concat` on the entire corpus generates an Out-Of-Memory (OOM) `MemoryError` and crashes the application server.

Implementation detail: The pipeline utilizes Python's `glob` library to identify all files matching `*_balanced_dataset.csv` . As each file is read (`pd.read_csv`), the pipeline utilizes Pandas fractional sampling:

```
df = pd.read_csv(f, engine='c')
df = df.sample(frac=0.03, random_state=42)
```

By sampling 3% (configurable via the `sample_frac` parameter) uniformly across all universities, the model preserves standard normal distributions and feature interactions while keeping the memory footprint under 200MB.

4.2 Feature Engineering and Semantic Mapping

The raw columns are transformed to meet the semantic naming conventions utilized by the existing dashboard logic.

- **`Term`** : Synthesized by concatenating `term_code` and `term_year` (e.g., "F 21").
- **`Dept_Code`** : Extracted positionally from the `section_id` .
- **`Student_Type`** : Safely mapped via `{"F": "Full-Time", "P": "Part-Time"}` .

4.3 Behavioral Economic Equations

The pipeline generates composite economic behavioral metrics:

1. **`Rental_to_Retail_Ratio`** : To prevent ZeroDivision errors when a book is inexplicably free, the denominator replaces `0.0` with `100.0` .

   ```
   retail_new_safe = retail_new.replace(0.0, 100.0)
   ratio = retail_rent / retail_new_safe
   mapped_df["Rental_to_Retail_Ratio"] = ratio.clip(0.0, 1.5)
   ```

2. **`Arbitrage_Index`** : Defined as `1.0 - Rental_to_Retail_Ratio` . A ratio of `0.2` implies the student can rent the book for 20% of the cost. This creates an Arbitrage Index of `0.8` (Extremely High), signaling massive savings outside the standard bundle and predicting a high likelihood of opt-out.

3. **`Wallet_Pressure_Score`** : The raw `price_affordability_score` is normalized to a percentage via Min-Max scaling.

   ```
   mapped_df["Wallet_Pressure_Score"] = (afford_score /
   afford_score.max()).clip(0, 1)
   ```

4. **`Digital_Lock_Flag`** : Derived directly from `ebook_ind` . Because digital access codes (like Pearson MyLab) cannot be bought on secondary markets, this is the strongest predictor enforcing an opt-in.

---

5. Machine Learning Engine (`feature_engine.py`)

The analytics engine doesn't rely on synthetic probability; it spins up a real inference pipeline using Scikit-Learn.

5.1 Algorithm Selection

We employ a `RandomForestClassifier` . Decision Trees natively handle missing data effortlessly and are mathematically robust to outliers in variables like `family_annual_income` . Trees also do not

require strict normalization unlike Logistic Regression.

Configuration:

```
clf = RandomForestClassifier(n_estimators=50, max_depth=5, random_state=42,
n_jobs=-1)
```

- `n_estimators=50` : Calculates the ensemble consensus over 50 specific trees.
- `max_depth=5` : Truncates tree growth. Ensures the model generalizes behavior over subsets rather than overfitting entirely to the training data.
- `n_jobs=-1` : Instructs the CPU to use all available multi-threading cores heavily decreasing fit times.

5.2 The Training Paradigm

1. **Feature matrix (X)** focuses entirely on 7 explicitly chosen regressors: `Arbitrage_Index` , `Wallet_Pressure_Score` , `Digital_Lock_Flag` , `Rental_to_Retail_Ratio` , `family_annual_income` , `is_rental` , and `has_scholarship` .
2. **Target array (y)** is `Actual_Purchase_Flag` (the absolute truth of whether a student bought the bundle).
3. The dataset drops `NaN` selectively across these columns to form `train_df` .

5.3 Forecast Output

After invoking `clf.fit(X, y)` , the pipeline calls `clf.predict_proba(X_all)` . It specifically isolates the probability mapping to the positive class (`1`), appending this array to the global dataframe as `Predicted_Purchase_Prob` .

The Projected Spend Algorithm: To turn these behavioral probabilities into capital execution, the engine runs:

```
Projected_Spend = Predicted_Demand_Units * Unit_Price * Predicted_Purchase_Prob
```

If a $100 book has a 30% probability of purchase, the projected spend allocation for that line item falls to $30.

5.4 Feature Importance Extraction

The code extracts internal Gini impurity measurements from the fitted classifier via `clf.feature_importances_` . This arrays perfectly alongside the feature names to form a secondary DataFrame (`fi_df`) explaining which factors are driving student behavior the most heavily (usually `Digital_Lock_Flag`).

---

6. Decision Support Dashboard (`dashboard_app.py`)

The presentation layer is powered entirely by Streamlit, heavily utilizing CSS injection for a specialized "glassmorphism" aesthetic targeting a deep navy background radially fading into black.

6.1 Performance Caching

To prevent the application from retraining the Random Forest every time a user toggles a UI filter, the system leverages `@st.cache_data` .

```python
@st.cache_data(show_spinner=False)
def get_data() -> tuple[pd.DataFrame, pd.DataFrame]:
    df = load_feature_table(sample_frac=0.03)
    df, fi = train_and_predict(df)
    return df, fi
```

This restricts the heavily intensive `pd.concat` and `clf.fit` routines to the initial server boot sequence.

6.2 Interactive Left Filter Menu

Users can dynamically constrain the entire dataset. The filter executes boolean masking:

```python
mask = pd.Series(True, index=df.index)
if term != "All":
    mask &= df["Term"] == term
```

Filtering updates the in-memory dataframe, instantly cascading layout repaints across all right-side charts.

6.3 Executive KPIs

The topmost row uses raw HTML `<div>` structures mapping to the `kpi_card` function.

- **Total Predicted Demand**: Flat summation of `Predicted_Demand_Units` across the filtered dataset.
- **Total Projected Spend**: Summation of `Projected_Spend` mathematically rounded to `Millions (M)` for readability.
- **High-Risk Opt-Out Rate**: Computes the mean of any `Opt_Out_Probability` strictly exceeding `0.6` (60%).

6.4 Advanced Data Visualizations

- **Price Sensitivity (Plotly Scatter)**: Maps `Rental_to_Retail_Ratio` on the X-axis against `Opt_Out_Probability` on the Y-Axis. Hovering over plots triggers tooltip arrays revealing underlying publishers. Visualizes exactly where the "tipping point" for pricing margins exists.
- **Feature Importance (Plotly Bar)**: A horizontal representation of the Gini impurity outputs fed explicitly from the ML script.
- **Format Preferences (Grouped Bar)**: Stacks physical vs. digital demand distinctly segmented by `Student_Type` (e.g. Part-Time students heavily lean towards Digital formats based on commuting friction).

---

7. Target Database Schema (`schema.sql`)

While the current Python application leverages Pandas over CSV execution, the architectural target maps to a rigid ANSI SQL schema.

1. **`FACT_ENROLLMENTS`** : Stores the absolute grain representing one row per student-course combination. Holds the `actual_purchase_flag` target label.
2. **`DIM_STUDENT_MASTER`** : Handles demographic normalization (`financial_condition` , `commuter_distance`).

3. **`DIM_ADOPTIONS`** : Separates textbook attributes to reduce data redundancy, handling `retail_new_price` , `is_digital` .
4. **`DIM_SECTIONS`** : Groups logical instances like courses, terms, modalities, and instructors.

---

8. Deployment and Execution

System Requirements

- Python 3.9+
- Memory: 4GB+ RAM (Due to Pandas internal fragmentation)

Packages

- `streamlit`
- `pandas`
- `numpy`
- `scikit-learn`
- `plotly`

Launch the Application

To run the server natively in development:

```
# 1. Activate the custom python virtual environment
source .venv/bin/activate

# 2. Start the Streamlit server
streamlit run dashboard_app.py
```

Upon execution, it will display a local WebServer URL (default: `localhost:8501`) and standard network execution URLs. The initial load heavily burdens the CPU for ~15 seconds while training the model against the 400k+ extracted rows; however, performance normalizes to near-0ms latency post-boot.